

Cloud-Based Movie Streaming Platform Using FastAPI and Next.js

Mustakim Ahmed Hasan (2421349042), MD. Nafiz Al Mamun (2232050642),
Tanim Ahmed (2312597042), Ahamed Osman Asif (2031080642)

CSE299.4; Group: 01

Department of Computer Science and Engineering
North South University

Abstract—This project proposes a full-stack cloud-based movie streaming platform developed as part of the CSE299 course. The system provides secure user authentication, movie discovery, adaptive video playback using HTTP Live Streaming (HLS), watch-history tracking, and recommendation endpoints. The architecture follows a decoupled frontend-backend model using Next.js, FastAPI, PostgreSQL, and Supabase Storage. The goal is to simulate real-world streaming infrastructure while gaining practical experience in scalable full-stack system design and deployment.

I. INTRODUCTION

Online streaming has become the dominant medium for media consumption. Modern platforms require secure authentication, scalable APIs, adaptive media delivery, and cloud-based deployment.

Unlike typical academic CRUD-based web projects, this system incorporates adaptive streaming (HLS), REST API architecture, typed schemas, cloud storage, and cross-origin deployment configuration. The objective is to build a simplified yet realistic streaming platform that mirrors industry-level architecture.

II. PROBLEM STATEMENT

Users require a centralized web application where they can:

- Register and authenticate securely
- Browse and search movie content
- Stream adaptive HLS video
- Resume watching from saved progress
- Receive content recommendations

Most classroom prototypes lack:

- Secure token-based session management
- Proper frontend-backend separation
- Cloud-ready deployment configuration
- Media streaming support

This project addresses these limitations by designing a production-style streaming architecture.

III. METHODS

The system follows a three-layer architecture:

A. Presentation Layer

The frontend is developed using Next.js with TypeScript. It implements:

- Responsive UI for desktop and mobile browsers
- SWR for data fetching and caching
- Tailwind CSS for styling
- HLS.js for adaptive video playback

B. Service Layer

The backend is built with FastAPI and structured using modular routers:

- `/auth` – user authentication endpoints
- `/movies` – movie catalog and recommendations
- `/history` – watch progress tracking

The backend uses asynchronous request handling and typed response schemas to ensure frontend-backend consistency.

C. Data Layer

- PostgreSQL stores user records, movie metadata, genres, subtitles, and watch history.
- Supabase Storage stores HLS playlists and media segments.

Key entities include User, Movie, Genre, Subtitle, and WatchHistory with appropriate relational mappings.

D. Deployment Approach

- Frontend and backend are deployed as separate cloud services.
- Environment variables manage database URLs, secret keys, and CORS origins.
- Media assets are hosted externally in object storage.

IV. EXPECTED OUTCOMES

The final system is expected to:

- Provide a fully functional streaming web application
- Successfully stream HLS media through cloud storage
- Maintain persistent authentication and watch history
- Demonstrate clean separation of concerns across layers
- Operate reliably under cloud deployment conditions

The platform will simulate real-world streaming infrastructure and showcase full-stack engineering competence.

V. RESOURCES

The project requires the following tools and technologies:

- Next.js (Frontend framework)
- FastAPI (Backend framework)
- PostgreSQL (Relational database)
- Supabase Storage (Cloud object storage)
- SQLAlchemy (ORM)
- HLS.js (Streaming support)
- GitHub (Version control)
- Cloud hosting provider for deployment

Official documentation sources:

- FastAPI Documentation: <https://fastapi.tiangolo.com>
- Next.js Documentation: <https://nextjs.org/docs>
- SQLAlchemy Documentation: <https://docs.sqlalchemy.org>
- Supabase Documentation: <https://supabase.com/docs>